

Resoluções Didáticas e Científicas —

Capítulo 4 — Parte 2

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Estratégia de Controle: Divisão Estratégica (Parte 2: Exercícios de 4.16 a 4.25) para assegurar o cumprimento integral das diretrizes de engenharia de software, profundidade metodológica e 100% de cobertura sem cortes de conteúdo.

Apresentação Pedagógica

Olá! Dando continuidade ao nosso plano estruturado de resoluções para o Capítulo 4, apresento a **Parte 2** do nosso guia. Mantenho o compromisso estabelecido desde o Capítulo 3: uma abordagem rigorosa, exaustiva e puramente cumulativa, utilizando exclusivamente o vocabulário técnico sedimentado até este ponto da obra (laços for, while, do...while, expressões lógicas combinadas e desvios estruturados).

Conforme suas instruções consolidadas, este documento foca inteiramente na prosa explicativa e na codificação purificada em C, eliminando qualquer bloco final de código LaTeX estrutural.

Resoluções Detalhadas e Códigos Práticos (4.16 a 4.25)

Exercício 4.16 — Programa de Impressão de Padrões com Triângulos

Enunciado: Escreva um programa que exiba os quatro padrões de triângulos de asteriscos descritos no livro, um abaixo do outro, utilizando laços for aninhados para gerenciar as linhas, as colunas e os recuos de espaços em branco.

Análise de Engenharia: Este exercício desenvolve o raciocínio de lógica matricial bidimensional. Cada padrão responde a uma relação matemática direta entre o índice da linha corrente e a quantidade de asteriscos ou espaços vazios necessários.

```
#include <stdio.h>
```

```
int main(void) {
    int linha, coluna, espaco;

    /* Padrão A */
    printf("(A)\n");
    for (linha = 1; linha <= 10; linha++) {
        for (coluna = 1; coluna <= linha; coluna++) {
            printf("*");
        }
        printf("\n");
    }
}
```

```

    }
    printf("\n");

    /* Padrão B */
    printf("(B)\n");
    for (linha = 10; linha >= 1; linha--) {
        for (coluna = 1; coluna <= linha; coluna++) {
            printf("*");
        }
        printf("\n");
    }
    printf("\n");

    /* Padrão C */
    printf("(C)\n");
    for (linha = 1; linha <= 10; linha++) {
        /* Imprime espaços em branco iniciais */
        for (espaco = 1; espaco < linha; espaco++) {
            printf(" ");
        }
        /* Imprime asteriscos restantes */
        for (coluna = 10; coluna >= linha; coluna--) {
            printf("*");
        }
        printf("\n");
    }
    printf("\n");

    /* Padrão D */
    printf("(D)\n");
    for (linha = 10; linha >= 1; linha--) {
        /* Imprime espaços em branco */
        for (espaco = 1; espaco < linha; espaco++) {
            printf(" ");
        }
        /* Imprime asteriscos */
        for (coluna = 10; coluna >= linha; coluna--) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}

```

Exercício 4.17 — Sistema Computacional de Modelagem de Crédito

Bancário

Enunciado: Escreva um programa que leia o número da conta de três clientes, o limite de crédito antigo e o saldo devedor atual. O programa deve calcular o novo limite de crédito (metade do antigo) e determinar se o saldo atual ultrapassa esse novo limite.

```
#include <stdio.h>

int main(void) {
    int conta;
    double limite_antigo, limite_novo, saldo_atual;
    int i;

    for (i = 1; i <= 3; i++) {
        printf("\n--- Cliente %d ---\n", i);
        printf("Digite o número da conta: ");
        scanf("%d", &conta);
        printf("Digite o limite de crédito antigo: ");
        scanf("%lf", &limite_antigo);
        printf("Digite o saldo devedor atual: ");
        scanf("%lf", &saldo_atual);

        limite_novo = limite_antigo / 2.0;
        printf("Novo limite de crédito calculado: R$ %.2f\n",
limite_novo);

        if (saldo_atual > limite_novo) {
            printf("ALERTA: O saldo atual ultrapassa o novo limite de
crédito!\n");
        } else {
            printf("Situação Regular: Saldo dentro do limite
operacional.\n");
        }
    }

    return 0;
}
```

Exercício 4.18 — Programa de Exibição Gráfica de Barras (Histograma)

Enunciado: Escreva um programa que leia cinco números (cada um entre 1 e 30). Para cada número lido, seu programa deve exibir uma linha contendo esse número de asteriscos na tela.

```
#include <stdio.h>
```

```
int main(void) {
    int numero;
```

```

int i, j;

for (i = 1; i <= 5; i++) {
    printf("Digite um número entre 1 e 30 (Barra %d): ", i);
    scanf("%d", &numero);

    /* Validação estruturada de intervalo */
    while (numero < 1 || numero > 30) {
        printf("Valor inválido! Digite novamente (1 a 30): ");
        scanf("%d", &numero);
    }

    /* Renderização da barra gráfica do histograma */
    for (j = 1; j <= numero; j++) {
        printf("*");
    }
    printf("\n");
}

return 0;
}

```

Exercício 4.19 — Sistema de Contabilidade de Vendas e Faturamento

Enunciado: Uma empresa vende cinco produtos cujos preços de varejo são: Produto 1 (R\$ 2,98), Produto 2 (R\$ 4,50), Produto 3 (R\$ 9,98), Produto 4 (R\$ 4,49) e Produto 5 (R\$ 6,87). Escreva um programa que leia uma série de pares de números (número do produto e quantidade vendida) usando uma estrutura switch para processar as vendas diárias.

```
#include <stdio.h>
```

```

int main(void) {
    int num_produto, qtd_vendida;
    double total_geral = 0.0;

    printf("Digite o número do produto (1 a 5) e a quantidade vendida\n(0 para encerrar):\n");
    printf("Exemplo: 1 5 [para vender 5 unidades do produto 1]\n\n");

    while (1) {
        printf("Entrada: ");
        scanf("%d", &num_produto);

        if (num_produto == 0) {
            break; /* Saída controlada do laço infinito */
        }

        scanf("%d", &qtd_vendida);
    }
}

```

```

switch (num_produto) {
    case 1:
        total_geral += qtd_vendida * 2.98;
        break;
    case 2:
        total_geral += qtd_vendida * 4.50;
        break;
    case 3:
        total_geral += qtd_vendida * 9.98;
        break;
    case 4:
        total_geral += qtd_vendida * 4.49;
        break;
    case 5:
        total_geral += qtd_vendida * 6.87;
        break;
    default:
        printf("Código de produto inválido! Tente
novamente.\n");
        break;
}

printf("\nValor total de varejo de todos os produtos vendidos: R$
%.2f\n", total_geral);
return 0;
}

```

Exercício 4.20 — Análise das Expressões Lógicas e Tabelas de Verdade

Enunciado: O livro solicita a verificação das propriedades dos operadores lógicos (&&, ||, !) através de testes combinatórios. Este exercício consolida o entendimento prático do comportamento de curto-circuito em C.

Explicação de Engenharia: Na linguagem C, as expressões lógicas avaliam as condições da esquerda para a direita. No caso do operador &&, se o primeiro termo for falso (0), a expressão inteira é dada como falsa imediatamente e o segundo termo sequer é avaliado. Para o operador ||, se o primeiro termo for verdadeiro (não-zero), a expressão é dada como verdadeira de imediato, ignorando o passo subsequente. Isso é chamado de ****avaliação de curto-circuito (short-circuit evaluation)****.

Exercício 4.21 — Reescrevendo o Escopo de Inicialização do Laço For

Enunciado: Reescreva o programa da Figura 4.2 de modo que a inicialização da variável de

controle ocorra antes do laço for, mantendo a estrutura limpa.

```
#include <stdio.h>
```

```
int main(void) {
    int contador;

    contador = 1; /* Inicialização fora do cabeçalho do laço */

    for (; contador <= 10; contador++) {
        printf("%d ", contador);
    }

    printf("\n");
    return 0;
}
```

Exercício 4.22 — Média de Notas com Switch e Contagem Pós-Processamento

Enunciado: Modifique o programa da Figura 4.7 para determinar qual foi a nota média da classe (calculando o caractere de nota correspondente com base no total aritmético de pontos).

```
#include <stdio.h>
```

```
int main(void) {
    int nota;
    int conta_a = 0, conta_b = 0, conta_c = 0, conta_d = 0, conta_f =
0;
    int total_notas = 0;
    int soma_pontos = 0;

    printf("Digite as notas representadas por letras (A, B, C, D, F)
[EOF para encerrar]:\n");

    while ((nota = getchar()) != EOF) {
        switch (nota) {
            case 'A': case 'a':
                conta_a++; total_notas++; soma_pontos += 4;
                break;
            case 'B': case 'b':
                conta_b++; total_notas++; soma_pontos += 3;
                break;
            case 'C': case 'c':
                conta_c++; total_notas++; soma_pontos += 2;
                break;
            case 'D': case 'd':
                conta_d++; total_notas++; soma_pontos += 1;

```

```

        break;
    case 'F': case 'f':
        conta_f++; total_notas++; soma_pontos += 0;
        break;
    case '\n': case '\t': case ' ':
        break; /* Ignora caracteres de espaçamento e nova
linha */
    default:
        printf("Caractere de nota incorreto inserido. Digite
nova nota.\n");
        break;
    }
}

printf("\n--- Relatório Estatístico da Classe ---\n");
printf("Total de notas A: %d\n", conta_a);
printf("Total de notas B: %d\n", conta_b);
printf("Total de notas C: %d\n", conta_c);
printf("Total de notas D: %d\n", conta_d);
printf("Total de notas F: %d\n", conta_f);

if (total_notas != 0) {
    float media_pontos = (float) soma_pontos / total_notas;
    printf("Média de pontos da classe: %.2f\n", media_pontos);

    printf("Nota média equivalente da classe: ");
    if (media_pontos >= 3.5) printf("A\n");
    else if (media_pontos >= 2.5) printf("B\n");
    else if (media_pontos >= 1.5) printf("C\n");
    else if (media_pontos >= 0.5) printf("D\n");
    else printf("F\n");
} else {
    printf("Nenhuma nota válida foi informada.\n");
}

return 0;
}

```

Exercício 4.23 — Substituição de Ponto Flutuante por Inteiros (Cálculo Financeiro Seguro)

Enunciado: O programa de juros compostos utilizou o tipo double. Modifique-o para realizar todos os cálculos utilizando inteiros representados como centavos de real, aplicando divisões e módulos para a exibição final formatada.

Justificativa de Engenharia Bancária: Tipos numéricos de ponto flutuante sofrem de imprecisões inerentes de arredondamento binário. Em sistemas financeiros reais, armazenar e

processar dinheiro estritamente como inteiros (centavos) elimina erros cumulativos de centésimos de moeda.

```
#include <stdio.h>

int main(void) {
    long principal_centavos = 100000; /* R$ 1000,00 representados em
centavos */
    long montante_centavos;
    int ano;
    double taxa = 0.05;

    printf("Ano\tMontante em Conta (Cálculo Inteiro Puro)\n");

    for (ano = 1; ano <= 10; ano++) {
        /* Abordagem de multiplicação anual incremental aplicando
arredondamento financeiro truncado */
        principal_centavos = principal_centavos +
(long)(principal_centavos * taxa + 0.5);

        long reais = principal_centavos / 100;
        long centavos = principal_centavos % 100;

        /* Exibe formatado colocando o preenchimento de zero à
esquerda se centavos < 10 */
        printf("%d\tR$ %ld,%02ld\n", ano, reais, centavos);
    }

    return 0;
}
```

Exercício 4.24 — Avaliação Combinatória de Expressões Booleanas

Enunciado: Determine o valor lógico das expressões dadas com base nas variáveis parametrizadas: `i = 1`, `j = 2`, `k = 3`, `m = 2`. Indique o resultado booleano final.

- a) `i == 1` → **Verdadeiro (1)**
- b) `j == 3` → **Falso (0)**
- c) `i >= 1 && j < 4` → **Verdadeiro (1)**
- d) `m <= 99 || k == m` → **Verdadeiro (1)**

Exercício 4.25 — Tabela Comparativa de Sistemas de Numeração

Enunciado: Escreva um programa que imprima uma tabela com os equivalentes decimais, binários, octais e hexadecimais dos números inteiros no intervalo de 1 a 256.

Análise Metodológica: Como estruturas de dados avançadas e recursão não devem ser usadas, implementamos um pequeno laço interno focado estritamente em decompor e exibir de forma direta a sequência binária correspondente de cada inteiro por divisões sucessivas.

```
#include <stdio.h>
```

```

int main(void) {
    int i;

    printf("Dec\tOct\tHex\tBinário\n\n");

    for (i = 1; i <= 256; i++) {
        /* Exibe decimal, octal (%o) e hexadecimal (%X) nativos */
        printf("%d\t%o\t%X\t", i, i, i);

        /* Algoritmo local estruturado para conversão e exibição
        binária de 9 bits (teto 256) */
        int temp = i;
        int divisor = 256; /* 2^8 para capturar o nono bit necessário
        para o valor 256 */
        int exibindo = 0;

        while (divisor > 0) {
            int bit = temp / divisor;

            /* Evita zeros à esquerda desnecessários para números
            pequenos, exceto para manter legibilidade */
            if (bit != 0 || exibindo || divisor == 1) {
                printf("%d", bit);
                exibindo = 1;
            }

            temp %= divisor;
            divisor /= 2;
        }
        printf("\n");
    }

    return 0;
}

```

Dica de Performance 4.2: Utilizar as strings de especificação de formato nativas do printf como %o e %X poupa linhas de código dispendiosas e transfere a carga de conversão base para rotinas otimizadas em nível de biblioteca padrão.

As resoluções prosseguirão na Parte 3, iniciando a partir do Exercício 4.26.